

How Parallax Was Born: The Intellectual Genesis of a Novel Framework

A Development Narrative

Bryan Alexander Buchorn · AMP

Independent Researcher

bryan.alexander@buchorn.com

March 2026

Research ideas rarely arrive fully formed. They emerge from constraints, from questions that refuse to stay simple, and from the moment when two things that were never supposed to connect suddenly click together. This document is a record of how Parallax came to be — from a live coding session on a knowledge graph visualizer to a novel theoretical framework for reasoning over structured knowledge. The path was neither straight nor planned.

What follows is a reconstruction of that path: the engineering problems that seeded it, the questions that pushed it forward, and the specific moments where the thinking made an unexpected leap.

1. Where It Started: A Visualization Problem

The story begins not in a research lab but inside AURA — a self-hosted AI assistant platform being built with a microservices architecture, a Neo4j knowledge graph, and a 3D graph visualization component built on Three.js and react-force-graph-3d. The knowledge graph was alive: every conversation, every ingested document, every retrieved fact was stored as nodes and relationships in Neo4j.

The visualization existed. Nodes floated in three-dimensional space, connected by edges representing semantic relationships. It was functional but static. The first request was deceptively simple:

"When I hit the clusters button, I want to see the clusters forming in real-time. Use the GPU if necessary."

— Bryan Alexander Buchorn, session initiation

That request — animate the clustering process as it happens — required thinking carefully about what clustering actually means in a live graph. The existing implementation used the Louvain algorithm, a greedy modularity optimizer that is fast but produces communities that can be internally disconnected. The first engineering decision was to replace it.

1.1 From Louvain to Leiden

The Leiden algorithm was the natural successor. Published by Traag, Waltman, and van Eck in 2019 as a direct correction to Louvain's known flaw, Leiden adds a refinement phase that guarantees every community in the final partition is internally well-connected. For a live

visualization where nodes were meant to physically attract toward their community centroid, a partition with disconnected communities would be visually incoherent — nodes would be pulled toward a centroid that did not represent their actual neighborhood.

Leiden was implemented via the `leidenalg` and `igraph` Python packages, integrated into the knowledge service as a `POST /communities/detect` endpoint. The frontend received community assignments over WebSocket and applied a custom d3-force cohesion force that computed per-community centroids at each simulation tick, attracting same-community nodes toward each other. The animation worked: clusters visibly flew together in real-time.

1.2 Adding Label Propagation

The next request was to offer a choice of algorithms. Label Propagation (LPA) was added as an alternative — `networkx.algorithms.community.label_propagation_communities`, fast and dependency-free. This introduced the first comparative question: how do Leiden and LPA actually differ?

The answer, worked through in the session, was structural:

- Leiden optimizes for global modularity — it finds the partition that maximizes the ratio of intra-community edges to what would be expected by chance. It is globally aware but can split locally coherent neighborhoods when global modularity favors a different partition.
- LPA propagates labels from local majority vote. Each node adopts the label most common among its neighbors. It is fast and locally coherent but suffers from the resolution limit: it can merge structurally distinct regions if they happen to be locally connected.

Both algorithms were exposed to the frontend as selectable options, along with a Hybrid mode that used LPA output as a warm-start for Leiden — LPA runs first, its partition is passed as `initial_membership` to Leiden, which then refines it for modularity. This was a known technique. It was not yet novel.

2. The Question That Changed Everything

The conversation about sequential algorithms — LPA followed by Leiden — prompted a question about refinement. Could communities be iteratively tightened after initial detection? A `/communities/refine` endpoint was added that accepted the current partition and ran Leiden from that starting point, tightening structure without a full reset.

But the more interesting question came immediately after:

"Can they be stacked or run one after another for further refinement, or does it completely redo the entire structure? Can the algorithm be refined to include structure from both?"

— Bryan Alexander Buchorn

The answer to the first part was already implemented: yes, they can be stacked, and the hybrid mode does exactly that. But the second part of the question — can the algorithm include structure from both simultaneously? — was different. That was not asking about a pipeline. It was asking about a fundamentally different kind of algorithm.

2.1 The Simultaneous Signal Hypothesis

The question was: instead of running LPA and then Leiden, what if a single algorithm computed both the LPA signal and the modularity gain signal at every individual node update, and made its decision based on both simultaneously?

This had not been done. The literature on community detection treats LPA and modularity optimization as separate paradigms. Hybrid approaches use one to initialize the other. No published algorithm computes both signals per-node and fuses them in a single update step.

The algorithm that emerged from working through this question became Dual-Signal Community Fusion (DSCF). Its core logic, derived in the session:

- For each node, compute the LPA signal: which community do most of my neighbors currently belong to? How confident is that vote?
- For each node, compute the modularity signal: which community move would produce the largest gain in global modularity? How large is that gain?
- If both signals agree on a move: execute it immediately. This is a consensus anchor — both local topology and global structure want the same change. High confidence.
- If only one signal says move: execute with a probability governed by that signal's confidence and the current temperature.
- If both signals disagree on which community to move to: execute a weighted probabilistic choice between the two targets, with weights governed by confidence and temperature.
- Anneal the temperature over iterations: early iterations are LPA-dominated (exploration), late iterations are modularity-dominated (exploitation).

A connectivity post-pass, borrowed from Leiden, splits any community whose induced subgraph is internally disconnected. The result is a partition where every community is locally coherent (LPA ensures this) and globally significant (modularity ensures this). A node ends up in its community because both signals agreed — not just one.

2.2 The Structural Duality

Working through the properties of DSCF communities revealed something important: they have a dual character that neither Leiden-only nor LPA-only communities possess.

Leiden communities are globally optimal but can be locally incoherent. LPA communities are locally coherent but can be globally naive. DSCF communities are both: the LPA component holds locally coherent groups together even when modularity would split them, and the modularity component prevents over-merging when LPA would absorb structurally distinct regions.

This dual property — short-range coherence combined with long-range structural significance — was interesting in its own right. But it became the foundation of something larger when the conversation took its most significant turn.

3. The Conceptual Leap: Knowledge Graphs as Language Models

After working through DSCF and establishing that it was a genuinely novel algorithm, the conversation shifted to a broader question. The framing was direct:

"Is that a new methodology? Question: How can we treat Knowledge Graphs like LLMs? Would the structure of an LLM be able to be adapted to the data?"

— Bryan Alexander Buchorn

This question reframed everything. The previous work had been engineering — better clustering, smoother animation, a novel algorithm. This was theoretical. It asked whether the two dominant paradigms for knowledge representation — explicit graphs and implicit neural weights — could be understood as structurally equivalent.

3.1 The Core Observation

The Transformer architecture derives its power from three mechanisms: multi-head attention (different heads specialize on different relational aspects of the input), deep composition (each layer builds on the previous, enabling multi-step reasoning), and positional awareness (the model knows where each token sits relative to others).

Working through this carefully, natural analogs appeared in graph structure for each of the three:

- Community structure maps to attention heads. Nodes within a community have strong mutual relevance. Communities specialize on different conceptual domains, just as different attention heads specialize on different relational aspects of a sequence.
- BFS hop depth maps to layer depth. Each traversal hop is one step of composed reasoning, just as each Transformer layer applies one round of attention-based transformation.
- Graph-structural features — PageRank (global centrality), betweenness (bridge importance), degree (connectivity) — map to positional encoding. They tell the model where each entity sits in the global information landscape.

The question was whether these were merely metaphors or whether they could be made operational. The answer was that they could — and the mechanism that makes them operational is Community-Structured Attention.

3.2 Community-Structured Attention

Graph Attention Networks (GATs) already exist. They apply learned attention weights between directly connected node pairs. But GATs have a fundamental limitation: attention is restricted to direct neighbors. A node can only attend to what it is already connected to. There is no global context.

The gap that CSA fills is precisely the gap between local GAT attention and global Transformer attention. CSA introduces a community membership term into the attention weight formula: nodes in the same DSCF community receive a base attention bonus, nodes in adjacent communities receive a smaller bonus, and nodes far across the community graph receive an exponentially decaying weight. This is global structural awareness, implemented without the $O(n^2)$ cost of full Transformer attention.

The full CSA weight for entity u attending to entity v at traversal hop k combines embedding cosine similarity, community membership score, edge type weight, normalized graph distance, and hop decay — five signals that together capture both the semantic and structural dimensions of relevance in a single scalar.

3.3 Why DSCF Communities Are the Right Attention Heads

The connection between DSCF and multi-head attention became clear when examining what trained Transformer attention heads actually specialize on. Research on attention head behavior in large language models has shown that different heads capture different kinds of relationships: some specialize on syntactic adjacency (local), some on semantic themes and coreference (long-range). The power of multi-head attention comes from this dual specialization operating simultaneously.

DSCF communities exhibit exactly this dual character structurally. The LPA component produces local coherence: nodes that are topologically proximate end up together. The modularity component produces global significance: communities correspond to structurally distinct regions of the graph. A DSCF community is present because both signals agreed — it is both locally tight and globally meaningful.

No previous community detection algorithm uses both signals simultaneously at the per-node level. And no previous graph reasoning system uses community structure as the organizing principle for attention. These two gaps, filled together, are what Parallax closes.

4. The Critical Distinction from Prior Work

Placing Parallax in context requires understanding precisely where it diverges from the closest existing systems. The most important comparison is Microsoft's GraphRAG.

4.1 What GraphRAG Does

GraphRAG builds a knowledge graph from documents, runs the Leiden algorithm to detect communities, and then generates natural language summaries of each community using an LLM. Those summaries are stored as text chunks and used as retrieval units in a RAG pipeline: when a query arrives, relevant community summaries are retrieved and passed to an LLM, which performs the actual reasoning.

GraphRAG is a retrieval system. The communities are used to chunk and summarize. The LLM reasons. The KG is passive.

4.2 What Parallax Does

Parallax uses communities as attention mechanisms, not as text chunks. There is no summarization step. The community structure is not converted to language — it is used directly to weight graph traversal. When a query arrives, Parallax performs beam-search traversal guided by CSA attention weights, which incorporate DSCF community membership. The output is a ranked list of reasoning paths: explicit sequences of (entity, relation, entity) triples that constitute the answer.

Parallax does not need an LLM to reason. An LLM can optionally be used to convert the traversal output to natural language — but the reasoning itself is performed by the graph, guided by community structure as attention. The KG is not passive. It reasons.

This inversion is the central claim: Parallax is not a better RAG system. It is a graph that reasons the way a language model reasons — through structured attention over community-organized knowledge — without being a language model.

5. The Role of AURA in the Genesis

AURA was not incidental to Parallax. Several components that will form Parallax's Phase 1 implementation already exist in AURA's codebase as a direct result of this session:

- The DSCF algorithm is implemented in Python in `services/knowledge_service/main.py`, depending only on `networkx`. It was written, reasoned about, and validated within AURA.
- The full algorithm suite — Leiden, LPA, Hybrid, and DSCF — is exposed via a unified `POST /communities/detect` endpoint with an `algorithm` parameter.
- The iterative refinement endpoint (`POST /communities/refine`) accepts an existing partition and tightens it — the operational equivalent of what Parallax calls progressive enhancement.
- The community broadcast WebSocket pattern — sending `community_detection_started` events to all connected clients before detection begins — is the real-time infrastructure that a live Parallax deployment would use.
- The Neo4j adapter patterns: Cypher query templates, bolt connection management, and entity/relationship schema conventions are all directly portable to Parallax's `neo4j_adapter.py`.

AURA served as the prototyping environment in which every core Parallax concept was first given code form. The spin-off is a generalization of what AURA proved was possible.

6. The Name

The name Parallax was chosen deliberately. In optics, parallax is the apparent displacement of an object when viewed from two different positions. The classic example is depth perception: the human visual system uses the parallax between the two eyes to compute distance information that neither eye alone can perceive. Two viewpoints on the same scene yield structural depth.

DSCF is parallax applied to graph community detection. LPA and modularity optimization are two viewpoints on the same graph. From the LPA viewpoint, a community is a set of nodes that locally reinforce each other's labels. From the modularity viewpoint, a community is a set of nodes whose internal connectivity exceeds random expectation. Neither viewpoint alone sees the full structure. Their combination — computed simultaneously at every node update — yields the structural depth that makes DSCF communities useful as attention heads.

The framework inherits the name because the same principle extends upward: a KG that reasons using community structure as attention is looking at knowledge from two angles simultaneously

— local topological coherence and global structural organization. That dual perspective is what enables multi-hop inference that is both grounded and interpretable.

7. What Makes This Novel

To be explicit about the claims:

- DSCF is a new community detection algorithm. Per-node simultaneous dual-signal fusion — LPA majority vote and modularity gain computed and fused at each individual node update — has not been published.
 - CSA is a new attention mechanism. The community membership term in the attention weight formula introduces global structural awareness into graph traversal without the $O(n^2)$ cost of Transformer attention and without the hard adjacency constraint of GATs.
 - The Transformer-to-KG equivalence table is a functional mapping, not an analogy. Every row in the table describes an operational substitution: the KG component can perform the same computational role as the Transformer component it maps to.
 - The attention head count adapts to the data. In Transformers, the number of heads is a hyperparameter. In Parallax, it is determined by the graph's own community structure. A graph with 70 natural communities has 70 attention heads.
 - The output is always a path. Every Parallax answer is a verified sequence of (entity, relation, entity) triples. The system cannot hallucinate a connection that does not exist in the graph.
-

8. What Comes Next

The white paper defines the theory. The AURA prototype proves the core algorithm works. What remains is to build Parallax as a standalone, framework-agnostic library and to validate the three central hypotheses on standard benchmarks (WebQSP, MetaQA-3hop).

The most important empirical question is H1: do DSCF communities produce better reasoning paths than Leiden-only or LPA-only communities as attention heads? The structural argument is strong. The proof is in the benchmark numbers.

The broader significance — if the hypotheses hold — is that a knowledge graph can reason through multi-hop questions with interpretable, verified paths, no training data, and no LLM required. In high-stakes domains where hallucination is unacceptable — medicine, law, cybersecurity — that property matters.

It started with wanting clusters to animate in real-time. It ended with a new way of thinking about what a knowledge graph can do.
